

Engine-Integrated Cel-Shading for Unreal Engine 5

Gabriel Paes Landim de Lucena
AKCIT-IMD
Natal, Brazil
Instituto Metr pole Digital
Universidade Federal do Rio Grande do Norte (UFRN)
Natal, Rio Grande do Norte, Brazil
gabriellucena23@gmail.com

Cesar Renn -Costa
AKCIT-IMD
Natal, Brazil
Instituto Metr pole Digital
Universidade Federal do Rio Grande do Norte (UFRN)
Natal, Rio Grande do Norte, Brazil
cesar@imd.ufrn.br

Charles Madeira
Instituto Metr pole Digital
Universidade Federal do Rio Grande do Norte (UFRN)
Natal, Rio Grande do Norte, Brazil
charles@imd.ufrn.br

Alyson Souza
AKCIT-IMD
Natal, Rio Grande do Norte, Brazil
Instituto Metr pole Digital
Universidade Federal do Rio Grande do Norte (UFRN)
Natal, Rio Grande do Norte, Brazil
alyson@imd.ufrn.br

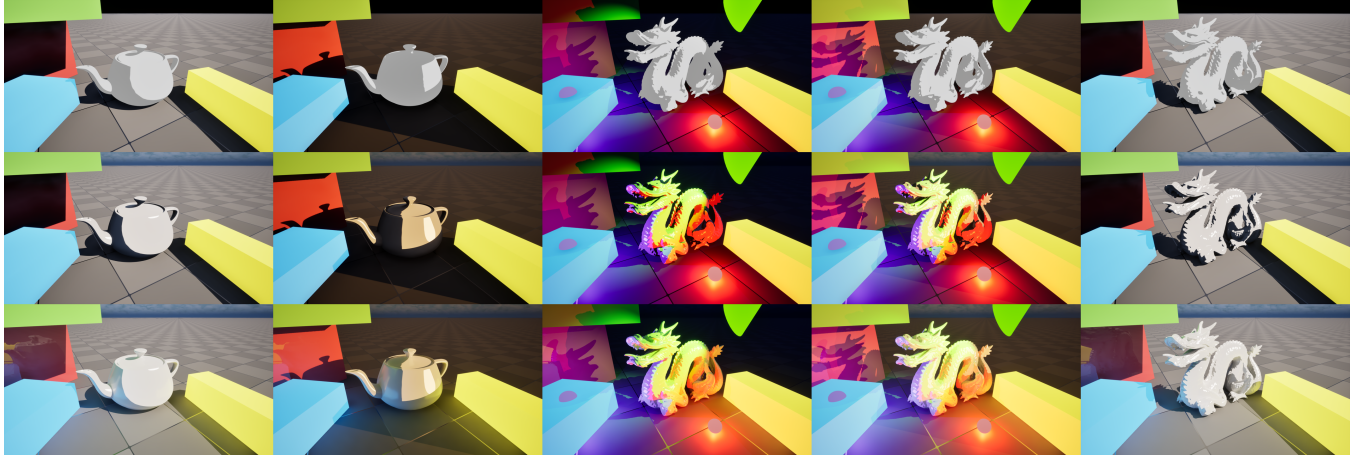


Figure 1: Teapot and Dragon rendered in different scenes with the toon shader. From top to bottom: Post-processing, our approach and our approach + ray-tracing.

Abstract

Implementing cel-shading in modern engines like Unreal Engine 5 is challenging, as traditional post-processing methods conflict with advanced features like ray tracing. This work presents a novel solution by modifying the engine’s source code, replacing the native Clear-Coat model with a custom cel-shader. Our approach uses a smoothstep function for artist-driven control and ensures full compatibility with UE5’s advanced rendering pipeline, including ray-traced reflections and dynamic lighting, overcoming the limitations of previous techniques.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA Posters ’25, Hong Kong, Hong Kong

  2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2134-2/25/12

<https://doi.org/10.1145/3757374.3771456>

CCS Concepts

• **Computing methodologies** → **Non-photorealistic rendering**; *Ray tracing*; **Rendering**; *Computer graphics*; • **Software and its engineering** → Context specific languages.

ACM Reference Format:

Gabriel Paes Landim de Lucena, Charles Madeira, Cesar Renn -Costa, and Alyson Souza. 2025. Engine-Integrated Cel-Shading for Unreal Engine 5. In *SIGGRAPH Asia 2025 Posters (SA Posters ’25)*, December 15–18, 2025, Hong Kong, Hong Kong. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3757374.3771456>

1 Introduction

Cel-shading is a non-photorealistic rendering (NPR) technique that emulates the stylized aesthetic of comics and cartoons. This is fundamentally achieved by applying hard quantized transitions to diffuse lighting results [Lake et al. 2000], using reflectance values to

sample a predefined texture [Barla et al. 2006], and adapting specular highlights to fit the stylized appearance [Anjyo and Hiramitsu 2003].

Although there are many implementations [Liao 2023], a robust and comprehensive toon shader remains a challenge within advanced renderers such as Unreal Engine. Common methods utilize post-process materials [Epic Games 2025], a technique that discards critical rendering data such as specular highlights, ray-traced contributions, and information from multiple light sources. This work, therefore, undertakes a deep investigation into implementing a cel-shader within Unreal Engine that directly addresses these limitations by fully supporting ray-tracing, multiple light sources, and specular highlights.

2 Methodology

Our implementation was developed using a source build of Unreal Engine 5. The core of our approach involves three distinct stages: first, isolating the lighting pipeline from data-altering pre-processing steps inherent to the original shading model; second, integrating our custom shader logic by replacing an existing shading model; and third, implementing a flexible, artist-driven lighting calculation for the cel-shaded effect.

2.1 Isolating the Lighting Pipeline from Data Pre-Processing

To ensure our custom lighting calculation uses unaltered data, we modified Unreal Engine's ClearCoat shading model at the source code level. We removed its physically-based pre-processing, including material layer blending and color remapping, to preserve the artist's exact base color.

Crucially, we also disabled the octahedral compression of normal vectors. While this technique optimizes memory, it can create artifacts in cel-shading, which relies on high-precision normals.

2.2 Integration via a Custom Shading Model

Instead of adding a new shading model from scratch, which can be a highly complex and invasive process, we opted to replace an existing one. The ClearCoat shading model was chosen as an ideal candidate as it is a specialized model with a narrower use-case (e.g., car paint), making its replacement less disruptive to general physically-based rendering workflows and as it provides access to "Custom Data" input pins in the material editor, designed to pass arbitrary floating-point data into the HLSL code.

The integration involved redirecting the shading model for ClearCoat within the engine's shader to our custom cel-shading function, allowing our logic to be executed within the engine's deferred rendering pipeline.

2.3 Customizable Gradient Shading Implementation

The core of the cel-shading effect is achieved through a custom lighting function implemented in HLSL. For each light affecting a surface, the luminance is calculated as the dot product between the surface's world normal ($\vec{N}$) and the light direction vector ($\vec{L}$), clamped to a $[0, 1]$ range : $\text{luminance} = \text{saturate}(\vec{N} \cdot \vec{L})$

Instead of using a hard step function to create discrete color bands, which can be inflexible, or a texture lookup, which adds overhead, we implemented a more versatile solution using the intrinsic HLSL smoothstep function. The `smoothstep(edge0, edge1, x)` function performs a smooth Hermite interpolation between 0 and 1 when x is between edge0 and edge1 . $\text{shading} = \text{smoothstep}(\text{Threshold}, \text{Threshold} + \text{Smoothness}, \text{luminance})$

By exposing Threshold (equivalent to edge0) and Smoothness (which defines edge1) as user-definable parameters passed through the "Custom Data" pins, we provide artists with a powerful and unified tool for controlling the visual style. A very small Smoothness value approximates a hard step function, creating the classic, sharp-edged cel-shading look. A larger Smoothness value produces a soft gradient, enabling a range of painterly styles. The same smoothstep-based calculation is applied consistently to diffuse lighting, specular highlights, and received shadows to ensure a coherent and unified visual aesthetic across the entire rendered image.

A test scene was created with varying lighting settings and a Newell teapot [Blinn and Newell 1976] and Dragon [Stanford Computer Graphics Laboratory 2003] 3D models to check if the shader is functioning correctly. As shown in 1, it is noticeable that compared to the post-processing shader, our approach presents with specular highlights, ray-tracing and multiple light support.

3 Conclusion

We have developed a high-fidelity cel-shader in Unreal Engine 5 by implementing a new shading model directly in the engine's source code. This novel approach surpasses the limitations of traditional post-processing methods by fully supporting advanced features like multiple dynamic lights, specular highlights, and ray-tracing. Our work successfully merges stylized, non-photorealistic rendering with modern physically-based pipelines, offering artists greater creative flexibility.

The primary limitation is the requirement of a custom engine build, which complicates distribution in team-based workflows. Future research will focus on creating a more accessible plugin-based architecture, applying this methodology to other NPR styles.

Acknowledgments

This work has been partially funded by the project Use of Immersive Technologies and AI for New Interfaces in Music and Arts supported by Advanced Knowledge Center in Immersive Technologies (AKCIT), with financial resources from the PPI IoT/Manufatura 4.0 / PPI HardwareBR of the MCTI grant number 057/2023, signed with EMBRAPPI.

Thanks to the Stanford Computer Graphics Laboratory for providing the Dragon 3D model used in testing.

References

- K.I. Anjyo and K. Hiramitsu. 2003. Stylized highlights for cartoon rendering and animation. *IEEE Computer Graphics and Applications* 23, 4 (2003), 54–61. doi:10.1109/MCG.2003.1210865
- Pascal Barla, Joëlle Thollot, and Lee Markosian. 2006. X-toon: an extended toon shader. In *Proceedings of the 4th International Symposium on Non-Photorealistic Animation and Rendering (Annecy, France) (NPAR '06)*. Association for Computing Machinery, New York, NY, USA, 127–132. doi:10.1145/1124728.1124749
- James F. Blinn and Martin E. Newell. 1976. Texture and reflection in computer generated images. *Commun. ACM* 19, 10 (Oct. 1976), 542–547. doi:10.1145/360349.360353

- Epic Games. 2025. Post Process Materials in Unreal Engine. <https://dev.epicgames.com/documentation/en-us/unreal-engine/post-process-materials-in-unreal-engine> Official Unreal Engine 5.6 documentation. Access date: 14/08/2025.
- Adam Lake, Carl Marshall, Mark Harris, and Marc Blackstein. 2000. Stylized rendering techniques for scalable real-time 3D animation. In *Proceedings of the 1st International Symposium on Non-Photorealistic Animation and Rendering* (Annecy, France) (NPAR '00). Association for Computing Machinery, New York, NY, USA, 13–20. doi:10.1145/340916.340918
- Junxin Liao. 2023. The Research of Cel-Shading in Non-photorealistic Rendering. In *2023 13th International Conference on Information Technology in Medicine and Education (ITME)*. 569–572. doi:10.1109/ITME60234.2023.00119
- Stanford Computer Graphics Laboratory. 2003. Stanford Dragon. 3D scanned model from the Stanford 3D Scanning Repository. Access date: 14/08/2025.